AP-Final

Bora Haliloglu

November 2025

1 Introduction

In this exam, we did tasks that, in the end, allowed us to simulate a multi-round, multi-agent variant of the Prisoner's Dilemma, under the metaphor of Galápagos finches helping each other clean their plumage (or not) of parasitic mites. All of the tests as requested are in the `Tests.hs` file. There are *67* test cases in total. These are written to test the most critical parts of the tasks. While *67* is not enough to test every edge case, it aims to cover a significant portion of it. The tasks are in a manner that values conciseness; libraries and functions are not imported unless they are essential.

2 Task A

In this task, I implemented the `Set`, `Union`, `Without`, and `Elem` operations in the APL evaluator. Each element of a set is evaluated left-to-right, and evaluation stops immediately on the first runtime error. Type checking ensures that `Union` and `Without` operate only on sets, while `Elem` requires its right-hand operand to be a set. To maintain deterministic results, element order is maintained and structural equality is used for membership and subtraction. I created 15 unit tests, 7 of which were positive and 8 of which were negative, to ensure accuracy. The positive tests include membership checks for both Boolean and numeric data types, unions, subtractions, and empty and nested sets. The negative tests verify that runtime mistakes like division by zero within set literals and non-set operands are handled correctly. Together, these tests verify the evaluator's value semantics, type enforcement, and appropriate error propagation.

3 Task B

In this task, the APL parser was extended to support the new syntactic constructs introduced in Task A: `Set`, `Union`, `Without`, and `Elem`. The grammar was modified to handle set literals enclosed in braces ({ }) and comma-separated elements, while maintaining correct operator precedence

and associativity. Both `union` and `without` were defined as *right-associative*, consistent with APL's recursive evaluation rules, whereas `elem` was parsed with higher precedence to ensure expressions such as `a elem S without T` are interpreted as `a elem (S without T)`. The parser eliminates potential conflicts with existing arithmetic and logical operators by distinguishing between binary operators and set literals by token context in order to prevent ambiguity. In this task I believe the solution I proposed, which was elegant, was in the *Union*, and *Without* parsing. The `Union` and `Without` are done right-associative as requested. This is done using the `<|>` operator and recursion.

Listing 1: Right Associativity of Union and Without

```
pUnionWithout :: Parser Exp
pUnionWithout = pExp1 >>= chain
  where
    chain x =
      choice
        [ do
            f <- (Union <$ lKeyword "union") <|> (Without <$ lKeyword
                 "without")
            y <- pUnionWithout
            chain $ f x y
          ,pure x
        ]
```

4 Task C

The Margolus block automaton's concurrent simulation server is implemented in this work, guaranteeing semantic equivalency with the sequential reference model. Every grid cell functions as a separate thread that solely uses message channels for communication, doing away with shared mutable state. Deterministic updates are enforced by the architecture using global barriers and synchronized phases. At initialization, the server spawns one thread per cell, each seeded with its initial state and Margolus position. Two phases are performed in each simulation step: Phase A handles horizontal interactions between (UL, UR) and (LL, LR) pairs, while Phase B handles vertical interactions (UL, LL) and (UR, LR). After both phases, all positions are shifted using the Margolus transform, matching the sequential `stepGrid` semantics.

Toroidal wrapping is applied to all neighbor lookups, ensuring periodic boundary conditions identical to `gridIndex` in the sequential model. The server uses global synchronization barriers to guarantee that all cells complete each phase before proceeding, thereby preserving the strict order of operations from the reference implementation. The `observeAutomaton` function retrieves each cell's state and applies `rulesObserve`, yielding the same

observable grid as `observeGrid`. Hence, the concurrent server is fully deterministic and behaviorally identical to the sequential simulator. I have initially tried to cover the cases where the grids are odd length as well. However, it is mathematically impossible to cover the plane in 2x2 blocks as there will some overlays and I could not see any specification that enforces me to handle overlays of the grids. Hence, I have made it so that there will a error that will be raised when the plane is of odd lenght in one dimension.

Listing 2: Odd Lenght Dimensions

```
1  when (odd h || odd w) $
2    ioError (userError $
3      "BlockAutomaton: cannot partition a "
4      ++ show h ++ "x" ++ show w
5      ++ " torus into disjoint 2x2 Margolus blocks. Both dimensions
           must be even.")
```

5 Task D

The `meet` function defines how two finches interact during a meeting according to the Galápagos rules. Each finch's suspended strategy is interpreted through a local helper `step`, which expects a `MeetOp` effect followed by either a `GroomOp` or `IgnoreOp`. Any deviation marks the finch as illegal. If both finches behave legally, their actions are combined using the payoff matrix from `Params`, producing HP deltas `(da, db)` that are added to their current HP values and clamped by `maxHP`. Each finch's continuation is then resumed with the partner's action, yielding updated strategies for future interactions.

If one finch performs an illegal action, the function returns `Nothing` for that finch and leaves the partner unmodified, and if both are illegal, it returns `(Nothing, Nothing)`.

6 Task E

This task implemented the configuration file parser for the Galápagos simulation. The parser reads model parameters, grid size, and strategies written in the extended APL syntax. The implementation supports both `height`–`width` and `width`–`height` orders, and allows parameters to be repeated, or be omitted; repeated definitions override earlier ones. Parameter updates are applied over `defaultParams` using functional composition.

Each `strategy` block is parsed with its name, hexadecimal colour (`#RRGGBB`), and an embedded APL expression enclosed in braces. All keywords are strictly matched and followed by optional whitespace, and invalid tokens, such as malformed colours or missing braces are rejected by Megaparsec's error reporting. The resulting configuration object correctly populates the `Config` structure with grid size, parameter values, and strategy

list. The only part that is not fully applying the specification file is the fact that the parser does not allow for the parts to appear in a random order.

7 Task F

In the context of the Galápagos, this task put into practice the simulation logic controlling finch interactions, reproduction, aging, and death. The rules make each grid cell either vacant or equipped with a finch by extending the block-cellular automaton architecture. The APL evaluator interprets the strategy expressions of neighboring finch pairs to determine how they interact at each step. The resulting behavior decides whether a finch survives, reproduces, or dies by updating its health points (HP) according to the payoff matrix. The design guarantees deterministic behaviour between the sequential and concurrent simulators. All operations respect the configuration parameters (`lifespan`, `maxHP`, `invDensity`, `invReproductionChance`, and payoff tuple) and have been verified by equivalence tests against the reference sequential model.

8 Conclusions From Simulation

After running the simulation, the grudger largely dominates the plane. The tit-for-tat or cold grudger can also be seen in some runs. This also proves the axiom that there is no single best strategy; there should be different strategies in many runs. The only strategy persistently appearing in many runs, however, is the Grudger. Moreover, the simulation handles the invalid cases successfully and has never resulted in a shutdown of the simulation. Therefore, we can confidently say that the strategy that is outperforming the rest is the Grudger.

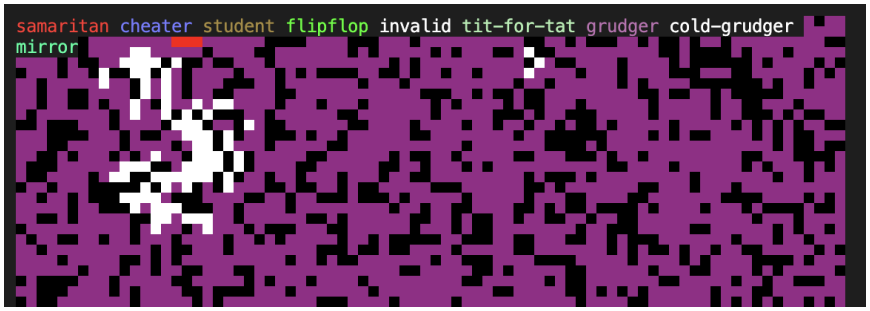


Figure 1: The Simulation After Some Epochs.

9 Questions

9.1 Question 1

$$
\begin{aligned}
\mathit{Exp} &::= \mathit{Eq} \\
\mathit{Eq} &::= \mathit{Elem}\ (\texttt{==}\ \mathit{Elem})^{*} \\
\mathit{Elem} &::= \mathit{UnionWithout}\ (\texttt{elem}\ \mathit{UnionWithout})^{*} \\
\mathit{UnionWithout} &::= \mathit{AddSub}\ \big((\texttt{union} \mid \texttt{without})\ \mathit{UnionWithout}\big)^{?} \\
\mathit{AddSub} &::= \mathit{MulDiv}\ \big((\texttt{+} \mid \texttt{-})\ \mathit{MulDiv}\big)^{*} \\
\mathit{MulDiv} &::= \mathit{Pow}\ \big((\texttt{*} \mid \texttt{/})\ \mathit{Pow}\big)^{*} \\
\mathit{Pow} &::= \mathit{LExp}\ (\texttt{**}\ \mathit{Pow})^{?} \\
\mathit{LExp} &::= \mathit{IfThenElse} \mid \mathit{Lambda} \mid \mathit{Let} \mid \mathit{ForLoop} \mid \mathit{App} \\
\mathit{App} &::= \mathit{Atom}\ (\mathit{Atom})^{*} \\
\mathit{Atom} &::= \mathit{Integer} \mid \mathit{Bool} \mid \mathit{Set} \mid \mathit{VName} \mid \texttt{(}\mathit{Exp}\texttt{)} \mid \texttt{groom} \mid \texttt{ignore} \mid \texttt{meet} \\
\mathit{IfThenElse} &::= \texttt{if}\ \mathit{Exp}\ \texttt{then}\ \mathit{Exp}\ \texttt{else}\ \mathit{Exp} \\
\mathit{Lambda} &::= \backslash\ \mathit{VName}\ \texttt{->}\ \mathit{Exp} \\
\mathit{Let} &::= \texttt{let}\ \mathit{VName}\ \texttt{=}\ \mathit{Exp}\ \texttt{in}\ \mathit{Exp} \\
\mathit{ForLoop} &::= \texttt{loop}\ \mathit{VName}\ \texttt{=}\ \mathit{Atom}\ \texttt{for}\ \mathit{VName}\ \texttt{<}\ \mathit{Exp}\ \texttt{do}\ \mathit{Exp} \\
\mathit{Set} &::= \{\ [\mathit{Exp}\ (\texttt{,}\ \mathit{Exp})^{*}]\ \} \\
\mathit{Bool} &::= \texttt{true} \mid \texttt{false} \\
\mathit{Integer} &::= \texttt{[0-9]}^{+} \\
\mathit{VName} &::= \texttt{[A-Za-z]}\ \texttt{[A-Za-z0-9]}^{*}
\end{aligned}
$$

The grammar is actually closely implemented in my parser. There is no left recursion, the associativity is per the specification and the *Union* and *Without* are right associative. The parser is built on top of the keywords. Meaning that the lower level is consisting of the keywords and the grammar and the rules are built on top.

9.2 Question 2

In my implementation when *Union* is applied repetitively to itself, memory consumption increases exponentially as the second part will be added to the end of the first after the first element is copied because of the $++$ operator. If x is of **L** lenght then after runs it will be **L, 2L, 4L...**

Listing 3: Odd Lenght Dimensions

```
1  eval env (Union e1 e2) = do
2    v1 <- eval env e1
3    v2 <- eval env e2
4    case (v1, v2) of
```

```
5       (ValSet s1, ValSet s2) ->
6         pure $ ValSet (s1 ++ s2)
7       (_, _) ->
8         failure "Union applied to non-sets"
```

9.3 Question 3

In my implementation, the *meet* function, the *finchStrategy* is calculated and then the computations continue. If for one reason the strategy diverges, the entire process will hang and will cause a deadlock. This is currently not solved in my code. However, it can be solved using a maximum tick counter. Each strategy has a maximum tick count it can consume. The tick will be decreased in every attempt in computation, if the tick reaches **0** the tick timeout branch will kick in then the strategy will be considered illegal. This will not disrupt the current flow of the code.

Listing 4: Deadlock From Infinite Loop

```
1     step self other =
2       case finchStrategy self of
3         Free (MeetOp k) ->
4           case k (finchID other) of
5             Free (GroomOp k2) -> Right (True, k2)
6             Free (IgnoreOp k2) -> Right (False, k2)
7             _ -> Left ()
8         _ -> Left ()
```

9.4 Question 4

While implementing the code, I encountered deadlocks mostly when the **2x2** grids were overlapping. This happened in cases where at least one of the dimensions were odd. I eliminated this case by throwing an error when the dimensions are invalid. I tried to eliminate deadlocks by reducing the circular wait schematics as much as I can. I also wait for *ack*'s before moving to another phase. This ensures that the system is not getting out of sync. However, there can still be some deadlocks. While waiting for *ack*'s if one cell crashes, there will be deadlock as the number of *ack*'s are not adding up.

9.5 Question 5

Listing 5: Single LOC for stepAutomaton

```
1   stepAutomaton (Automaton srv) = sendTo srv MsgStep
```

A single message sent to the message box of the server. Requesting a step.

Listing 6: Server Loop Initiating Phases

```
broadcast (DoPhase PhaseA) (aIn st)
waitNAcks (aAcks st) (h*w)
broadcast (DoPhase PhaseB) (aIn st)
waitNAcks (aAcks st) (h*w)
```

Listing 7: Phase A

```
writeChan (wInbox pCh) (Req s)
Res sP <- readChan (eInbox myCh)
...
writeChan (wInbox pCh) (Res sP')
...
Req _ <- readChan (wInbox myCh)
writeChan (eInbox (chOf (west h w (i,j)))) (Res s)
Res sNew <- readChan (wInbox myCh)
```

Informing all cells to start Phase A and Phase B. The broadcast is sequential but the cells perform concurrently.

Listing 8: Phase B

```
writeChan (nInbox pCh) (Req s)
Res sP <- readChan (sInbox myCh)
...
writeChan (nInbox pCh) (Res sP')
...
Req _ <- readChan (nInbox myCh)
writeChan (sInbox (chOf (north h w (i,j)))) (Res s)
Res sNew <- readChan (nInbox myCh)
```

Listing 9: Acknowledgements

```
writeChan acks Ack
```

Listing 10: Observation

```
writeChan cellIn (Pull r)
s <- readChan r
```

$$N = h \times w \qquad \text{(number of cells)}$$

Messages per step:
$$\begin{aligned} M_{\text{total}} &= 2N_{\text{broadcast}} + 3N_{\text{pairwise}} + 2N_{\text{ack}} + 1_{\text{client}} \\ &= (2+3+2)N + 1 \\ &= \boxed{7N+1} \end{aligned}$$

Concurrency:

$$\begin{cases} \text{Broadcast messages:} & \text{sequentially sent, concurrent effect} \\ \text{Pairwise (Req/Res/Res') messages:} & \text{fully concurrent} \\ \text{Acknowledgements:} & \text{concurrent from all cells} \end{cases}$$

As it can be seen there is no major effect in terms of concurrency, minimal sequential logic is used.

9.6 Question 6

The memory of a finch is stored in their *Strategy*. To inherit memory from the parent in our case means that we need to pass that onto the child. So in the *reproduce* function, we can use the parents continuation and this would essentially make the child inherit the memory. However, this would reduce the diversity. Another point would be strategies that include having a memory of previous finch interactions would cause some newborn finches to be aggressive towards finches that they didn't have an interaction with. This would make a hostile and biased environment.

9.7 Question 7

This can easily be tested by changing the parameters arbitrarily in the parsed file and checking if the configuration is correctly handled. In my implementation, arbitrary orders are not handled but repeated parameters are handled.